Blockchain Data Availability: Benchmarking Throughput, Economics, Utilization, and Security of Six Production Protocols

Prasad Kumkar[a,*], Pranjal Sarode[a]

[a]*Chainscore Labs, Pune, Maharashtra, India*

Abstract

Data availability (DA) limits how far blockchains can scale. This paper presents a benchmarking study of six production DA protocols (Polkadot ELVES, Ethereum EIP-4844/PeerDAS, Celestia, Espresso Tiramisu, NEAR, and Avail) using 90 days of mainnet data (November 2025 to February 2026). We introduce a taxonomy that classifies DA architectures into three paradigms: data availability sampling (DAS), verifiable information dispersal (VID), and validator-set recovery. Under a unified methodology we measure throughput (MiB/s), cost ($/MiB), utilization, and block dynamics across all six protocols. Key findings are: (i) mean utilization stays below 40% for every protocol, so current measurements are demand-constrained rather than supply-constrained; (ii) median DA cost spans more than eight orders of magnitude, from nano-dollars (Espresso) to tens of cents per MiB (NEAR); (iii) no protocol slashes for pure data withholding, so enforcement is indirect in every design; and (iv) no protocol guarantees permanent availability: protocol-defined retention ranges from ∼25 hours (Polkadot) to ∼18 days (Ethereum), while Avail and Espresso leave retention to node operators. Ethereum's Fusaka upgrade and two blob-parameter-only forks fall inside the window and serve as natural experiments on fee-market behaviour. We release all collection tooling, datasets, and analysis code under open licenses.

Keywords: data availability, blockchain scalability, erasure coding, data availability sampling, benchmarking, rollups, distributed systems, measurement

1. Introduction

A blockchain can scale only if every participant can be confident that block data has been published and can be retrieved. This property, *data availability* (DA), is a security requirement for rollups, parachains and sharded designs. Without it, validators cannot verify state transitions, light clients cannot detect fraud, and the system's trust model reduces to that of a permissioned committee [1].

Several DA designs have reached production in the past three years. Ethereum introduced blob-carrying transactions (EIP-4844) and, with the Fusaka upgrade in December 2025, activated PeerDAS for cell-level sampling [2, 3]. Celestia launched the first dedicated DA network with two-dimensional erasure coding and namespace-filtered sampling [4]. Polkadot's ELVES protocol distributes erasure-coded parachain data across its validator set with approval-gated finality [5]. Espresso's Tiramisu introduces a three-layer Verifiable Information Dispersal (VID) architecture that explicitly rejects DAS in favor of algebraic dispersal proofs [6]. NEAR provides sharding-based DA through Nightshade with a rollup-oriented blob store [7], and Avail combines KZG polynomial commitments with DAS-enabled light clients on a Substrate-based chain [8].

Published comparisons of these designs are qualitative surveys [1, 9], vendor material, or measurements of a single protocol [10, 11]. We are not aware of a study that measures throughput, cost and utilization across several DA networks with one methodology, so rollup developers choosing a DA backend have no common empirical baseline to work from.

This paper makes the following contributions:

1. A taxonomy that classifies DA protocols by verification paradigm (data availability sampling, verifiable information dispersal, or validator-set recovery), encoding scheme and commitment type (Section 2).
2. A comparison of encoding redundancy, commitment schemes (KZG, NMT and Merkle) and verification mechanisms across the six protocols, based on their specifications and documentation (Section 3).
3. Measurements from 90 days of mainnet data (November 2025–February 2026) for Polkadot, Ethereum, Celestia, Espresso, NEAR and Avail, covering throughput (MiB/s), cost ($/MiB), utilization and block dynamics under common definitions (Section 5).
4. A comparison of trust assumptions, Byzantine fault tolerance thresholds, withholding enforcement and retention policies (Section 6).
5. The collection scripts, raw datasets and analysis code, released under Apache 2.0 (code) and CC BY 4.0 (data).

*Corresponding author.

Email addresses: prasad@chainscorelabs.com (Prasad Kumkar), pranjal@chainscorelabs.com (Pranjal Sarode)

2. Background and Related Work

2.1. Data Availability Fundamentals

Data availability (DA) must be distinguished from the related but distinct problems of data *storage* and data *retrieval*. Storage concerns the persistence of data over time; retrieval concerns the ability to read stored data on demand. DA addresses a different question: *was the data published at all?* Formally, a block producer who withholds part of a block's data can cause validators and light clients to accept an invalid state transition, because the withheld data may contain a fraudulent transaction whose invalidity can never be proven if the data is unavailable [12].

Definition 1 (Data Availability)**.** *A block B satisfies the data availability property if every honest participant can retrieve the complete block data within a bounded time window Δ after the block header is published.*

A DA scheme must provide four properties:

1. **Completeness.** All data constituting block B is published to the network. A DA scheme is complete if no honest node accepts a block header unless the corresponding data has been made available to the network.
2. **Soundness.** The encoding of B is verifiably correct. Given a commitment $C(B)$ included in the block header, any participant can verify that a received coded piece is consistent with $C(B)$ without possessing the entire block.
3. **Availability detection.** Resource-constrained participants (light clients) can assess whether the data behind a block header is available without downloading the full block. This property separates DA from simple data retrieval.
4. **Reconstruction.** The original data B can be recovered from any sufficiently large subset of coded pieces. If $E(B)$ denotes the erasure-coded encoding of B producing n coded pieces with redundancy factor r, then B is reconstructible from any n/r pieces.

Notation. Throughout this paper we use the following conventions. A block's raw data is denoted B, with $|B|$ its size in bytes. The encoding function $E(B)$ produces n coded pieces (variously called shares, chunks, or cells depending on the protocol). A commitment $C(B)$ is included in the block header and serves as a binding digest of $E(B)$. Verification $V(C, s_i)$ checks that coded piece s_i is consistent with commitment C. The redundancy factor $r = n \cdot |s_i|/|B|$ captures the total storage expansion introduced by the encoding.

2.2. Taxonomy of DA Approaches

We classify the six protocols along three dimensions: the *availability verification paradigm* (how clients gain confidence that data is available), the *encoding scheme* (how raw data is expanded into coded pieces: 1D Reed–Solomon [13], 2D Reed–Solomon, or polynomial VID), and the *commitment scheme* (how pieces are bound to the header: KZG [14], Namespaced Merkle Trees, or plain Merkle trees). Section 3 compares encodings and commitments in detail; here we define the three verification paradigms.

DAS (Avail, Celestia, Ethereum/PeerDAS). Light clients request a few random coded pieces; if all arrive with valid proofs, the client concludes with high probability that the full data is available. Security rests on an *honest-minority* assumption: at least one honest, non-eclipsed full node must seed correctly encoded pieces into the network.

Validator-set recovery (Polkadot, NEAR). The validator set collectively stores erasure-coded chunks and attests to availability through consensus (Polkadot's availability bitfields [5], NEAR's per-shard chunk producers [7]); there is no client-side sampling.

VID (Espresso). Verifiable information dispersal [15] combines erasure coding with algebraic proofs that each committee member received a valid piece; a threshold of signatures forms a DA certificate [6]. Light-client confidence derives from the certificate, not from sampling.

2.3. Protocols Under Study

We summarize each protocol as deployed during our window (November 2025–February 2026); Table 3 gives the parameters.

Polkadot (ELVES). The relay chain erasure-codes parachain Proof-of-Validity (PoV) data across ~600 validators [5, 16]; a block is available once two-thirds sign availability bitfields, after which it enters approval checking and GRANDPA finality. Up to 100 cores each carry a PoV of up to 10 MiB every 6 s. DA is priced through coretime rather than per byte, and chunks are kept for ~25 hours.

Ethereum (EIP-4844 and PeerDAS). Blobs of 128 KiB are committed with KZG and priced by a separate exponential blob-gas market [2]. Pectra (EIP-7691 [17]) set a target/maximum of 6/9 blobs per block. The Fusaka upgrade on 3 December 2025 activated PeerDAS (EIP-7594 [3]), with 2× column extension over 128 columns, and EIP-7918 [18], which bounds the blob base fee from below by a fraction of the execution base fee. Two blob-parameter-only (BPO) forks then raised the target/maximum to 10/15 (9 December) and 14/21 (7 January 2026). Blobs are pruned after 4,096 epochs (~18 days).

Celestia. A $k \times k$ original data square is extended to $2k \times 2k$ and committed with Namespaced Merkle Trees, enabling namespace-filtered retrieval [4]. CometBFT gives single-slot finality every ~6 s. App version 6 activated on 24 November 2025, and on 3 December the maximum square rose from 128×128 (8 MiB) to 256×256 (32 MiB). Light nodes sample within a 7-day window (CIP-036).

Espresso (Tiramisu). Tiramisu layers a KZG-based VID scheme (Savoiardi), a DA committee for fast retrieval (Mascarpone), and an untrusted CDN for bulk delivery [6]; it is now marketed as EspressoDA. HotShot gives responsive finality of ~2 s and the maximum block size was 1 MB throughout our window. Mainnet 0 runs 100 nodes operated by 20 parties without slashing. Retention is set per node: pruning is off by default and, when enabled, targets 7 days.

NEAR. Nightshade shards erasure-code chunk data among each shard's chunk producers [7]. Rollups post data as function-call input to a blob-store contract; it is pruned after the 5-epoch garbage-collection window (∼1.5 days at the ∼0.6 s block time in force since May 2025). A single transaction is capped at 1.5 MiB. There is no DAS and no active withholding challenge.

Avail. Row-wise 1D Reed–Solomon encoding with KZG commitments supports DAS by light clients over a DHT [8]. BABE/GRANDPA produce a block every 20 s; application-specific data retrieval lets clients download only their application's data. The maximum block size was 4 MiB throughout the window. Nodes prune *state* after 256 blocks but keep block data by default, so there is no protocol-defined DA expiry; the VectorX bridge relays DA attestations to Ethereum with a ∼2-hour delay.

2.4. Related Work

Foundational theory. Croman et al. [19] quantified the throughput and latency limits of monolithic blockchains, motivating the separation of data publication from execution. Al-Bassam et al. [12] introduced fraud and data availability proofs, the foundation for DAS: by erasure-coding block data and letting light clients randomly sample coded pieces, a network can guarantee availability with high probability under an honest-minority assumption. Hall-Andersen, Simkin, and Wagner formalized the security definition of DAS [20] and proposed FRIDA, a DAS construction without a trusted setup [21]. Coded Merkle Trees [22] and the ACeD oracle [23] reduce the cost of detecting incorrect coding. On the dispersal side, asynchronous verifiable information dispersal [15] is the primitive behind committee-based designs, and Nazirkhanova, Neu, and Tse [24] adapted it to rollups using KZG commitments [14]; Fisch et al. [25] extend VID to permissionless settings.

Protocol-specific designs. Al-Bassam's LazyLedger proposal [4] introduced a dedicated DA blockchain with namespace-filtered retrieval, which evolved into Celestia. Burdges et al. [5] formalized Polkadot's ELVES protocol for availability and validity, proving security under Byzantine assumptions with approval-based checking. Bearer et al. [6] describe the HotShot consensus and Tiramisu DA architecture. Ethereum has progressed from blob transactions (EIP-4844 [2]) through blob throughput increases (EIP-7691 [17]) to peer-level DAS (EIP-7594 [3]), an incremental path toward Danksharding [26]; Chaudhuri et al. [27] and Pigaglio et al. [28] study the networking design space of Ethereum DAS through simulation.

Empirical and economic studies. Several recent studies measure Ethereum's blob market in isolation. Crapis et al. [29] model the economics of EIP-4844; Heimbach et al. [10] and Park et al. [30] analyse blob fee dynamics and the impact of EIP-4844; and Lee et al. [11] measure blob utilisation over 180 days. All of these focus on Ethereum alone.

Table 1: Encoding and redundancy.

Protocol	Encoding	Redund.	Reconstruction
Polkadot [5]	1D RS (validators)	∼3×	f+1 of n (n=3f+k)
Ethereum [3]	1D RS (PeerDAS cells)	∼2×	50%+ of 128 columns
Celestia [4]	2D RS ($k \times k \to 2k \times 2k$)	∼4×	k×k shares
Espresso [6]	Polynomial VID	$1/r$, r=m/n	m of n evaluations
NEAR [7]	1D RS (chunk parts)	∼3×	1/3 of parts
Avail [8]	1D RS (row-wise)	∼2×	n of 2n coded pieces

Surveys. Saif et al. [1] and Thibault et al. [9] survey DA and rollup designs, classifying approaches by trust assumptions and encoding strategies, without measurements from production networks.

Positioning of this work. To our knowledge, this paper is the first to measure six production DA protocols simultaneously under one methodology. Prior empirical work covers Ethereum alone, and prior comparative work is qualitative or simulation-based. Our study complements the theoretical results above and the surveys of [1, 9] by providing a cross-protocol empirical baseline, and it is the first to observe Ethereum's DA market across the Fusaka (PeerDAS) and BPO transitions.

3. Protocol Architecture Comparison

We compare the six protocols on encoding and redundancy, commitment scheme, and verification paradigm. These choices fix a protocol's bandwidth cost, what a light client can check on its own, and which parties must be trusted.

3.1. Encoding and Redundancy

All six protocols use Reed–Solomon (RS) erasure coding, but they differ in dimensionality, redundancy factor and how data is reconstructed. Table 1 summarizes these differences.

Higher redundancy costs proportionally more bandwidth and aggregate storage; in return, data can be recovered when a larger fraction of coded pieces is missing.

Celestia has the highest redundancy at ∼4× due to its two-dimensional RS structure: original data of dimension $k \times k$ is extended to $2k \times 2k$, yielding four times the original data volume in shares [4]. The overhead pays for two-dimensional sampling, in which light clients sample from both the rows and the columns of the extended matrix. The 2D layout also allows a single row or column of length $2k$ to be recovered from any k of its shares, so a node can repair one row without reconstructing the whole block.

Polkadot and NEAR both use ∼3× redundancy, distributing erasure-coded pieces across their respective validator sets. In Polkadot's ELVES protocol, parachain block data (Proofs of Validity, or PoVs) are RS-encoded and distributed such that each validator stores one coded chunk, with reconstruction requiring f+1 of n chunks where $n = 3f + k$ [5, 16]. NEAR's Nightshade sharding similarly encodes chunk data with ∼3× redundancy, requiring only 1/3 of the parts for reconstruction [7]. In both, responsibility for the data is spread over the validator set; there is no separate committee and no sampling.

Table 2: Commitment scheme properties (KZG: Avail, Ethereum, Espresso; NMT: Celestia; Merkle: Polkadot, NEAR).

Property	KZG	NMT	Merkle
Proof size	$O(1)$, ~48 bytes	$O(\log n)$	$O(\log n)$
Trusted setup	Yes (SRS)	No	No
Namespace filter	No	Native	No
Verification	Pairings	Hashes	Hashes
Encoding correctness	Algebraic	Fraud proofs	Not verifiable

Ethereum and Avail have the lowest redundancy, ~2×. Ethereum's EIP-4844 blob transactions encode data into 128-byte cells arranged across 128 columns. PeerDAS extends each blob's polynomial evaluation points such that any 64 of the 128 columns suffice for reconstruction [3]. Avail performs row-wise RS encoding on its application data matrix, extending n original pieces to $2n$ coded pieces per row, so that any n of the $2n$ pieces suffice for recovery [8]. With a 2× extension any half of the coded pieces recovers the data, which together with KZG commitments is enough for DAS.

Espresso's Tiramisu uses a polynomial-based verifiable information dispersal (VID) scheme. Data is represented as a polynomial that is evaluated at n distinct points (one per committee member); any m evaluations suffice for interpolation, where the code rate $r = m/n$ is configurable [6]. Algebraically this is an RS code, but because it is built into the VID commitment layer, one proof certifies both that the encoding is correct and that each member received its piece.

3.2. Commitment Schemes and Proof Systems

The commitment scheme determines what a light client can verify, whether a trusted setup is needed, and the cost of producing and checking proofs. The six protocols use three families: KZG polynomial commitments, Namespaced Merkle Trees (NMTs) and standard Merkle trees. Table 2 compares their properties.

KZG polynomial commitments [14] are used by Avail, Ethereum, and Espresso. A KZG commitment to a polynomial $p(x)$ of degree d is a single elliptic curve group element (~48 bytes on BLS12-381), and an opening proof for any evaluation $p(z) = v$ is also a single group element. Verification requires two pairing operations, which are computationally more expensive than hash evaluations but yield $O(1)$-size proofs regardless of the data size. KZG commitments enable *algebraic verification of encoding correctness*: given commitments to the rows (or columns) of an erasure-coded data matrix, anyone can verify that the extended portions are consistent with the original data by checking polynomial relationships between commitments, without accessing the underlying data. This lets Avail and Ethereum light clients reject incorrectly encoded data without fraud proofs. The drawback is that KZG needs a structured reference string (SRS) from a trusted setup; anyone holding the setup's secret could forge proofs. Ethereum and Avail both ran multi-party ceremonies to reduce this risk, Ethereum's with over 140,000 contributors.

Celestia uses Namespaced Merkle Trees (NMTs) [4]. An NMT is a standard Merkle tree augmented with namespace identifiers: each leaf is tagged with a namespace, internal nodes carry the minimum and maximum namespace of their subtrees, and the tree is sorted by namespace. This structure enables *namespace inclusion and exclusion proofs*. A light client can request all shares belonging to a specific namespace (e.g., a particular rollup's data) and receive a Merkle proof that the returned set is complete (no shares from that namespace have been omitted, and no extraneous shares have been included). Rollup nodes use this to download only their own data instead of the whole block. NMT proofs are $O(\log n)$ in size. Encoding correctness, however, is *not* algebraically verifiable from the NMT alone. Instead, Celestia relies on Bad Encoding Fraud Proofs (BEFPs) to detect and propagate evidence of incorrect erasure coding [12].

Polkadot and NEAR use standard Merkle trees. Polkadot commits to erasure-coded PoV chunks via a Merkle trie whose root is included in the relay chain block header [16, 31]. NEAR similarly uses Merkle roots to commit to chunk data within shard blocks. Merkle inclusion proofs are $O(\log n)$ and need only hashing to verify. They give light clients no way to check encoding correctness or to sample: a proof shows that a piece belongs to the committed set but says nothing about whether the erasure coding was done correctly. In both Polkadot and NEAR, encoding correctness is enforced through the validator set (validators who reconstruct and re-encode can detect errors), but this verification is not accessible to light clients or external observers.

3.3. Verification Paradigms: DAS, VID, and Validator Recovery

The protocols differ most in how availability is verified. The three paradigms below differ in their trust assumptions and in what a light client can check.

3.3.1. Data Availability Sampling (DAS)

DAS, as formalized by Al-Bassam et al. [12], enables light clients to verify data availability without downloading the full block. A light client randomly requests s coded pieces from the network; if all s are returned, the client gains probabilistic confidence that the full data is available. Against a $(1-q)$-withholding adversary (one that has made fraction q of the data unavailable), a client sampling s independent pieces achieves confidence $1 - q^s$.

Celestia was the first production network to deploy DAS. Its 2D RS structure means that light clients sample from the $2k \times 2k$ extended square. Because an adversary must withhold at least ~25% of the extended square to prevent reconstruction, each sample detects withholding with probability $\geq 1/4$, so 16 random samples give a Celestia light client approximately $1 - (3/4)^{16} \approx 99\%$ confidence that the data is available, assuming that at least one honest full node is non-eclipsed and participates in the peer-to-peer network [4].

Avail implements DAS over its 1D RS-encoded rows with KZG commitments. Avail light clients sample random cells from the data matrix; each cell is accompanied by a KZG opening proof. With 10 random samples, an Avail light client

achieves $1-(1/2)^{10} \approx 99.9\%$ confidence $(1-1/1024)$ [8]. Because each cell is checked against the committed row polynomial, the client also verifies encoding correctness without fraud proofs. Avail light clients fetch cells from a Kademlia DHT run by light clients and fall back to full-node RPC; cells fetched over RPC are written back into the DHT, so later requests can be served by other light clients.

Ethereum activated DAS through PeerDAS (EIP-7594) in the Fusaka upgrade on 3 December 2025. Under PeerDAS, each blob is extended from 64 to 128 evaluation points (columns), and each validator is responsible for custody of a subset of columns determined by its node ID. In the Fulu specification every node custodies at least 4 of 128 custody groups (`CUSTODY_REQUIREMENT`), nodes with attached validators custody at least 8 plus one per 32 ETH of balance (`VALIDATOR_CUSTODY_REQUIREMENT`), and every node samples 8 columns per slot (`SAMPLES_PER_SLOT`) [3]. Each node samples only a few columns, so confidence comes from the aggregate: with custody spread over ~1,000,000 validators, each of the 128 columns is very likely held by at least one honest validator. PeerDAS also introduces a "supernode" role: nodes that subscribe to all 128 data column subnets and can reconstruct the full data matrix from partial columns. The Fulu networking specification requires at least one supernode on the network for reconstruction of missing data, an assumption that did not exist before PeerDAS, when every consensus node downloaded all blob data [3].

DAS on all three protocols relies on two assumptions: (i) at least one honest, non-eclipsed full node must hold the complete data and respond to sampling requests, and (ii) sufficient sampling participation must exist in the network to ensure that withholding is detected before the data propagation window closes. An adversary capable of eclipsing a light client's peer connections can cause the client to falsely conclude that data is available (or unavailable) regardless of the true state.

3.3.2. Validator-Set Recovery

In Polkadot and NEAR the validator set stores and serves the erasure-coded data, and availability is attested through consensus instead of sampling.

Polkadot's ELVES protocol ties availability to finality [5]. After a parachain collator produces a block candidate, the relay chain validators erasure-code the Proof of Validity (PoV) and distribute one chunk per validator. Each validator that successfully receives and stores its chunk signs an *availability bitfield* (a bitmap indicating which PoV chunks it holds). A PoV is considered available when $\geq 2/3$ of validators have signed its bitfield. The GRANDPA finality gadget will not finalize a relay chain block unless the availability bitfields for all included parachain candidates have reached the $2/3$ threshold [16, 31]. Under the $2/3$ honest-majority assumption the guarantee is deterministic: unavailable data cannot be finalized.

NEAR's Nightshade sharding distributes chunk data across shard validators (chunk producers). Each chunk producer is responsible for producing and serving the chunk parts for its assigned shard. NEAR's availability enforcement relies on a "lazy signer" mechanism with probabilistic accountability: validators are required to attest to chunk availability, but enforcement is primarily through the consensus protocol's liveness requirements rather than explicit sampling [7]. A challenge mechanism exists in the block structure for disputing invalid or unavailable chunks, but as of our observation period, this mechanism is "not used today" in practice.

Because availability evidence is a precondition for finality (Polkadot) or block production (NEAR), validators have a direct incentive to store and serve data. Light clients, however, cannot check availability themselves: a Polkadot or NEAR light client must trust that the $\geq 2/3$ supermajority attested honestly, with only the validators' stake behind that trust.

3.3.3. Verifiable Information Dispersal (VID)

Espresso's Tiramisu replaces DAS with algebraic dispersal proofs [6]. It has three layers:

- **Savoiardi (VID layer):** Data is encoded as a polynomial and evaluated at n points (one per committee member). A vector commitment (using KZG) binds the evaluations to the polynomial commitment, producing a compact proof that each committee member received a correct evaluation. This provides *bribery resistance*: even if an adversary bribes a threshold of committee members, the algebraic binding prevents them from claiming to hold data they do not possess.
- **Mascarpone (certification layer):** Committee members who receive and verify their evaluations produce threshold signatures attesting to availability. A DA certificate is formed when $\geq 2/3$ of the committee has signed, providing a compact ($O(1)$-size) availability attestation.
- **Cocoa (CDN layer):** A secondary retrieval path through an untrusted CDN that provides availability even if committee members go offline, with cryptographic proofs linking the fallback data to the original VID commitments.

Confidence is deterministic: there is no sampling, and a block is finalized only with a DA certificate. DAS detects withholding probabilistically, whereas Savoiardi's algebraic binding makes it infeasible for a committee member to claim falsely that it holds its piece.

Retrieval, however, requires cooperation from the committee or the Cocoa CDN. There is no mechanism for an independent light client to sample data and verify availability without relying on the committee's attestations. Espresso argues that DAS adds "unnecessary redundancies" given VID's algebraic guarantees [6]; as a result, retrievability depends on the committee and the CDN.

3.3.4. No Slashing for Data Withholding

None of the six protocols slashes for pure data withholding. Each has some way to detect or discourage it, but enforcement is always indirect:

- **Polkadot:** Validators are slashed for casting invalid *validity* votes (attesting that an invalid parachain block is valid),

but there is no slashing for failing to store or serve erasure-coded chunks. A validator that withholds its chunk simply does not sign the availability bitfield, potentially delaying but not corrupting finality [5].

- **Celestia:** Bad Encoding Fraud Proofs (BEFPs) allow any full node to prove that the block producer performed incorrect erasure coding [12]. However, BEFPs address encoding correctness, not data withholding. A block producer that correctly encodes data but subsequently withholds it faces no direct slashing penalty.
- **Ethereum:** PeerDAS enforces availability through peer scoring and disconnection: validators that fail to serve their custody columns receive negative gossip scores and may be disconnected from the p2p network. There is no protocol-level slashing for custody failures [3].
- **Espresso:** In the current Mainnet 0 deployment, enforcement is reputational only. No slashing mechanism is implemented for committee members that fail to serve their VID evaluations [6].
- **NEAR:** A challenge mechanism exists in the block structure that would allow validators to dispute chunk unavailability, but this mechanism is "not used today" in the production network. Enforcement relies on the consensus protocol's liveness requirements [7].
- **Avail:** Slashing is implemented for equivocation (double-signing) but not for data withholding. Validators that fail to make sampled cells available face no direct economic penalty [8].

Proving that a node *has* data but *refuses* to serve it is harder than proving that it served *incorrect* data. Withholding is an omission fault, and attributing an omission to malice instead of a network partition, latency or node failure is an open problem in distributed systems.

Table 3 collects these parameters. DAS protocols (Celestia, Avail, and Ethereum with PeerDAS) let light clients verify availability themselves but depend on network-level assumptions; validator-recovery (Polkadot, NEAR) and VID (Espresso) designs give deterministic guarantees that hold only if the validators or committee are honest. The commitment scheme follows from the paradigm: KZG gives Ethereum, Avail and Espresso algebraic encoding checks, Celestia pairs sampling with NMTs and fraud proofs, and plain Merkle trees suffice where validators check the encoding themselves. All six use the classical $f < n/3$ Byzantine threshold but apply it differently: Polkadot gates finality on it, Ethereum spreads custody, Celestia relies on sampling participation, Espresso on committee certification, NEAR on per-shard honesty, and Avail on sampling backed by algebraic commitments.

4. Methodology

4.1. Data Collection Framework

We collect per-block production data from six DA protocols over a 90-day observation window (November 15, 2025 through February 14, 2026). For each protocol, we develop custom collection scripts that interface with mainnet RPC endpoints or explorer APIs, extracting block-level metrics at the granularity needed for throughput and cost analysis. All collectors are resumable: interrupted runs checkpoint progress in per-day CSV files and resume from the last collected block.

Table 4 summarizes the data sources, block counts, and collection methods for each protocol.

For each protocol, the collector captures the following:

Polkadot. We scan relay chain blocks via the Polkadot.js RPC API, extracting three event types per block: `paraInclusion.CandidateBacked`, `paraInclusion.CandidateIncluded`, and `paraInclusion.CandidateTimedOut`. We also parse the `paraInherent.enter` extrinsic to extract signed availability bitfield counts and per-core availability fractions. Throughput is computed as an upper bound: included × maxPovBytes/cadence, since the relay chain does not expose actual PoV sizes (only commitments). Cost data is collected separately from the Coretime chain, scanning `broker.Purchased`, `broker.Renewed`, and `broker.SaleInitialized` events, as well as relay chain `onDemand.OnDemandOrderPlaced` events.

Ethereum. We collect per-block header fields (`blob_gas_used`, `excess_blob_gas`) from execution-layer RPC endpoints across the full 90-day window, which spans four regimes: Pectra (target 6, max 9), Fusaka (same blob parameters, plus PeerDAS and the EIP-7918 fee floor, from block 23,935,694 on December 3, 2025), BPO1 (target 10, max 15, from block 23,975,778 on December 9), and BPO2 (target 14, max 21, from block 24,179,383 on January 7, 2026). Era boundaries are the first blocks at or after each fork's activation slot. The blob base fee is computed from the header's `excess_blob_gas` using the EIP-4844 exponential pricing formula with the update fraction of the active era. ETH/USD prices are sampled hourly from CoinGecko.

Celestia. Block data is retrieved from the Celenium indexer API, which provides 22 columns per block including `blobs_size`, `blobs_count`, `square_size`, `fill_rate`, and `fee_utia`. The window spans app version 5 and version 6 (from 24 November 2025), and a maximum-square increase from 128 to 256 (8 to 32 MiB) on 3 December 2025; we compute capacity with the maximum in force on each day.

Espresso. We fetch all blocks from the Espresso query-service explorer API using binary search to locate the start height and 20 concurrent workers fetching 100 blocks per page (~1,630 blocks/s collection rate). Each block record contains `size_bytes`, `num_transactions`, and `block_time_ms`. Since HotShot uses responsive consensus, block times are variable (no fixed cadence). The base fee has been verified as 1 wei/byte since genesis across headers from height 1K to 10.3M.

Table 3: Feature and parameter comparison of the six DA protocols during the observation window.

Feature	Polkadot	Ethereum	Celestia	Espresso	NEAR	Avail
Encoding	1D RS (validators)	1D RS (PeerDAS)	2D RS ($k \to 2k$)	Poly. VID	1D RS (chunks)	1D RS (rows)
Commitment	Merkle trie	KZG	NMT	KZG + vector	Merkle	KZG
Redundancy	~3×	~2×	~4×	Configurable	~3×	~2×
DAS support	No	Yes (PeerDAS)	Yes (2D)	No (VID)	No	Yes (1D)
Light client DAS	No	Custody	Yes (16 samples)	No	No	Yes (10 samples)
Block time	6 s (relay)	12 s	6 s	~2 s	~0.6 s	20 s
Max DA capacity	10 MiB×100	2.63 MiB (BPO2)	8→32 MiB*	~1 MiB	4 MiB×9	4 MiB
Protocol max MiB/s	~167	~0.22	1.33→5.31*	~0.37	~59	~0.21
Finality	12–60 s	~13 min	~6 s	~2 s	~2 s	20–60 s
DA retention	25 h	~18 d	7 d (sampling)	Node-set	~1.5 d	Node-set†
Fee model	Weight-based	Blob market	Gas-based	Per-byte	Gas-based	Weight-based
Withholding	Finality gate	Peer scoring	BEFPs (encoding)	Reputational	Unused challenge	None
Byzantine threshold	$f < n/3$	$f < n/3$	$f < n/3$	$f < n/3$	$f < n/3$/shard	$f < n/3$

*Celestia's maximum square rose from 128 to 256 (8 to 32 MiB) on 3 Dec 2025. †Avail prunes state after 256 blocks, but block data is kept by default (Substrate `archive-canonical` block pruning).

Table 4: Data Collection Summary Across Six DA Protocols (90-Day Window)

Protocol	Source	Blocks	Key Fields	Language	Price Source
Polkadot	Relay chain RPC	~1.3M	candidates backed/included, bitfields, core index	TypeScript	CoinGecko (DOT)
Ethereum	Execution + Beacon RPC	~648K	blob_gas_used, excess_blob_gas, blob count	Python/SQL	CoinGecko (ETH)
Celestia	Celenium API	~1.3M	blobs_size, square_size, fill_rate, fee_utia	Python	CoinGecko (TIA)
Espresso	Query-service API	~2.9M	size_bytes, num_transactions, block_time_ms	Python	CoinGecko (ETH)
NEAR	NEAR Lake S3 + RPC	~7.8M	total_encoded_bytes, gas_used, gas_price	TypeScript	CoinGecko (NEAR)
Avail	Substrate RPC	~389K	submit_data_bytes, block_fee_plancks	TypeScript	CoinGecko (AVAIL)

NEAR. We employ a dual-source strategy: NEAR Lake S3 for bulk historical collection (~275 blocks/s at concurrency 100) and NEAR RPC as fallback (~2.5 blocks/s). Per-block fields include `num_shards`, `total_encoded_bytes` (sum of chunk encoded lengths), `total_gas_used`, and `gas_price`.

Avail. The collector uses the Polkadot.js API (Avail is Substrate-based) to extract `submit_data_bytes`, `submit_data_count`, and `block_fee_plancks` (from `TransactionFeePaid` events) per block. Runtime version (`spec_version`) is tracked to account for parameter changes.

4.2. Metric Definitions

We apply four groups of metrics to every protocol.

4.2.1. Throughput (MiB/s)

We distinguish three throughput measures:

- **Protocol maximum**: the theoretical ceiling given current parameters. Protocol-specific formulas:

$$T_{\max}^{\text{Polkadot}} = \frac{\text{effective_cores} \times \text{max_pov}}{\text{cadence}} \quad (1)$$

$$T_{\max}^{\text{Ethereum}} = \frac{\text{max_blobs} \times 128\ \text{KiB}}{\text{slot_time}} \quad (2)$$

$$T_{\max}^{\text{Celestia}} = \frac{\text{max_data_bytes}}{\text{avg_block_time}} \quad (3)$$

  where $\text{effective_cores} = \min(\text{num_cores}, \lfloor \text{validators}/5 \rfloor)$ and cadence is 6s (async backing) for Polkadot.

- **Observed throughput**: measured from production data as

$$T_{\text{obs}} = \frac{\sum \text{payload_bytes}}{\Delta t \times 2^{20}} \quad \text{(MiB/s)} \quad (4)$$

  aggregated at hourly and daily granularity.

- **Utilization**: $U = T_{\text{obs}}/T_{\max}$, representing the fraction of available DA capacity in use.

4.2.2. Cost ($/MiB)

The protocols price DA in different units, which we normalize as follows:

- **Per-byte fee markets** (Celestia, Espresso, Avail): cost is directly computable from on-chain fee data and token prices.

$$\text{cost_per_MiB} = \text{fee_per_byte} \times 2^{20} \times \text{token_USD} \quad (5)$$

- **Blob gas market** (Ethereum): cost follows EIP-4844 exponential pricing:

$$\text{cost_per_MiB} = \frac{\text{blobBaseFee}_{\text{gwei}} \times 2^{20}}{10^9} \times \text{ETH_USD} \quad (6)$$

- **Coretime pricing** (Polkadot): DA is bundled with execution capacity. We compute:

$$\text{cost_per_MiB} = \frac{\text{coretime_price_DOT} \times \text{DOT_USD}}{\text{max_MiB_per_region}} \quad (7)$$

for bulk coretime, and fee_paid/max_pov_MiB for on-demand. This yields a *best-case* cost assuming full PoV utilization every block.

- **Gas + storage staking** (NEAR): upload cost is gas-based (burned); state storage requires locking 1 NEAR per 100 KB. Rollup DA blobs avoid state costs by using function call inputs (pruned).

We report spot cost at percentiles (p50, p90, p99) and volume-weighted average price (VWAP) over the 90-day window.

4.2.3. Annualized Cost

Where a protocol drops DA data after a fixed window, keeping the data available means reposting it periodically. We define:

$$\text{cost_per_MiB_year} = \text{cost_per_MiB} \times \frac{365}{\text{retention_days}} \quad (8)$$

This normalization applies only where the protocol defines a retention window (Polkadot 25 h, NEAR ~1.5 d, Celestia 7 d sampling window, Ethereum ~18 d); Avail and Espresso leave retention to node operators, so we report their spot cost only.

4.2.4. Block Dynamics

We measure block time distributions (p10, p50, p90), block size distributions, and empty block rates. For protocols with variable block times (Espresso's responsive HotShot consensus), we report the empirical distribution rather than a fixed value.

4.3. Normalization and Comparability

Comparing these protocols raises six methodological issues:

1. **Demand-constrained measurements**: mean utilization is below 40% for every protocol in the window, so observed throughput measures demand. We therefore report protocol maximum throughput alongside it.
2. **Token price volatility**: USD-denominated costs are sensitive to token prices, which fluctuated during our window (e.g., ETH ranged \$2,000–\$4,000; TIA ranged \$0.30–\$0.85). We use contemporaneous hourly prices from CoinGecko rather than fixed spot rates, and report VWAP alongside percentile distributions.
3. **Pricing units**: Polkadot sells blockspace (coretime), Ethereum prices blob gas, and Celestia and Espresso charge per byte. We convert all to \$/MiB, but Polkadot's coretime price also buys execution, so its per-MiB figure is a lower bound on pure DA cost.
4. **Upper-bound throughput for Polkadot**: the relay chain does not expose actual PoV sizes on-chain (only commitments). Our throughput measurement uses included_candidates × max_pov_size, which is an upper bound. Actual throughput is lower since real parachains typically use 1–3 MiB per candidate.
5. **Protocol upgrades within window**: Ethereum underwent three upgrades (Fusaka, BPO1, BPO2) and Celestia transitioned eras during our observation period. We segment analysis by protocol era where applicable and treat these transitions as natural experiments (Section 5.4).
6. **Throughput measurement conventions**: Two definitional choices affect comparability with other sources (e.g., L2Beat). First, for Ethereum we report protocol maximum throughput using the *absolute maximum* blob count per block (21 under BPO2), whereas some trackers use the EIP-4844 *target* blob count (14 under BPO2), below which fees trend toward zero. The target is the fee-market equilibrium and the maximum is the hard protocol ceiling. Second, for Celestia we measure *user data throughput* (the original data square, ODS), not the full 2D Reed–Solomon encoded square. The extended square is 4× larger (a $k \times k$ ODS becomes $2k \times 2k$ after encoding), so sources that report total encoded bandwidth will show approximately 4× our figures. We report user data throughput because it reflects the amount of application-level data that can be posted per unit time.

4.4. Reproducibility

All collection scripts, raw datasets, analysis notebooks, and generated figures are published in a public repository (`https://github.com/Chainscore/score-da`; archived at `https://doi.org/10.5281/zenodo.23276229`) under Apache 2.0 (code) and CC BY 4.0 (data) licenses. Docker environments encapsulate dependencies for each collector. Per-protocol `chain_config.json` files record exact protocol parameters, governance milestones, and era boundaries active during collection. Analysis is implemented in Python (pandas, matplotlib); scripts regenerate all figures and tables from the raw CSV data. Dune dashboards for each protocol are linked from the repository README.

5. Empirical Results

5.1. Observation Window and Protocol Eras

Our window (November 15, 2025 to February 14, 2026) contains several parameter changes that we treat as natural experiments:

- **Ethereum**: four regimes: Pectra (target 6/max 9 blobs), Fusaka (same blob parameters, plus PeerDAS and the EIP-7918 fee floor, from Dec 3), BPO1 (target 10/max 15, from Dec 9), and BPO2 (target 14/max 21, from Jan 7).
- **Celestia**: app version 6 from Nov 24, and a maximum-square increase from 128 to 256 (8 to 32 MiB per block) on Dec 3.
- **Polkadot, Espresso, NEAR, Avail**: no major parameter changes.

Table 3 lists the active parameters.

Table 5: 90-day throughput and utilization summary.

Protocol	Max MiB/s	p50 MiB/s	Mean MiB/s	Mean util.
Polkadot	~167	~42.8†	~43.0†	~25.8%
Ethereum	0.22	~0.051	~0.051	~35%
Celestia	1.33→5.31	~0.018	~0.024	~1.7%
Espresso	0.37	~0.002	~0.003	~1.5%
NEAR	~59	~0.107	~0.108	~0.18%
Avail	0.21	~0.001	~0.002	<1%

†Upper bound: assumes max PoV size per included candidate. Ethereum and Celestia utilization is against the maximum in force on each day.

Table 6: 90-day cost summary ($/MiB).

Protocol	p50	p90	p99	VWAP	Pricing
Polkadot	5.3×10^{-6}*	2.7×10^{-5}	3.5×10^{-5}	—	Coretime
Ethereum	7.3×10^{-3}	0.023	0.074	0.18	Blob gas
Celestia	0.020	0.030	0.042	0.025	Gas/byte
Espresso	3.1×10^{-9}	3.4×10^{-9}	3.5×10^{-9}	3.1×10^{-9}	Fixed fee
NEAR	0.53	0.71	0.92	0.54	Gas
Avail	0.0096	0.032	1.05	0.0074	Fee+modifier

*Sparse: 14 of 94 days had bulk coretime purchases; amortized over max PoV capacity. Quantiles are over the distribution of daily median costs in the window for every protocol; VWAP is total DA spend divided by total DA volume.

5.2. Throughput and Utilization

Fig. 1(a) overlays 7-day rolling throughput for all six protocols on a logarithmic axis, with dashed lines at each protocol maximum; Fig. 1(b) shows utilization on a shared linear axis. Table 5 gives aggregate statistics.

No protocol's mean utilization exceeds 40%. Ethereum's is the highest: ~60% under Pectra and Fusaka, falling to ~31% after BPO1 and ~23% after BPO2 as capacity grew faster than demand. Polkadot is stable at ~25–27%, with ~26 of 100 cores engaged per day. Celestia runs at ≤1% in steady state, and Espresso and Avail at a few percent at most. With spare capacity everywhere, observed MiB/s shows how much each network is used and says little about what it could carry. NEAR has the lowest utilization (~0.18%) yet the second-highest volume: ~833 GiB over the window, against Polkadot's upper-bound ~340 TiB, Ethereum's ~392 GiB and Celestia's ~200 GiB. It carried ~9 GiB of payload a day, but its ~59 MiB/s capacity across 9 shards is far larger.

5.3. Cost Analysis

Fig. 2 shows per-protocol daily cost quantile bands (p10/p50/p90) and Table 6 summarizes the distributions.

Median DA cost spans more than eight orders of magnitude, from ~$\$3\times10^{-9}$/MiB on Espresso to ~$0.53/MiB on NEAR, and much of that range comes from differences in economic design. Espresso's fee has been fixed at 1 wei/byte since genesis, so its cost follows ETH/USD, falling from ~$\$3.5\times10^{-9}$ to ~$\$2\times10^{-9}$ as ETH fell; in the permissioned Mainnet 0 it is a placeholder and carries no information about demand.

Before Fusaka, Ethereum's blob base fee sat at the 1-wei floor on ~28% of blocks and the median cost was ~$\$4\times10^{-9}$/MiB. From the first day after Fusaka no block is at the floor and the median rises six orders of magnitude to ~$0.009/MiB. BPO1 had not yet added capacity at that point, so the change comes from EIP-7918 (Section 5.4).

Table 7: Annualized DA cost, accounting for retention.

Protocol	Retention	Reposts/yr	p50 $ /MiB/yr
NEAR	~1.5 d	~243	~129
Celestia	7 d	~52	~1.0
Ethereum	~18 d	~20	~0.15
Polkadot	25 h	~350	~0.002
Avail	node-set	—	(spot 0.0096)
Espresso	node-set	—	(spot 3×10^{-9})

Avail's median ($0.0096/MiB) is close to Celestia's, but its p99 reaches ~$1.05 during brief activity surges around December 4 and in late December, which suggests that its fee modifier amplifies volatility even at low throughput. NEAR's cost fell from ~$0.75 to ~$0.30/MiB over the window and is the highest among the active fee markets, because general gas pricing is applied to encoded receipt data. Polkadot's cost is observed on few days and is negligible, because coretime pays for execution and DA together.

5.3.1. Retention-Adjusted Cost

Table 7 and Fig. 3(b) normalize cost by retention (Section 4.2).

Short retention multiplies cost: NEAR's 5-epoch garbage-collection window (~1.5 days) requires ~243 reposts per year and gives by far the highest annualized cost (~$129/MiB/yr), while Celestia's 7-day sampling window lifts its $0.02/MiB spot cost to ~$1/MiB/yr. Ethereum's 18-day window keeps its annualized cost near $0.15/MiB/yr, and Polkadot's cost stays negligible even after ~350 reposts. Avail and Espresso define no protocol-level expiry, so their cost cannot be annualized on the same basis.

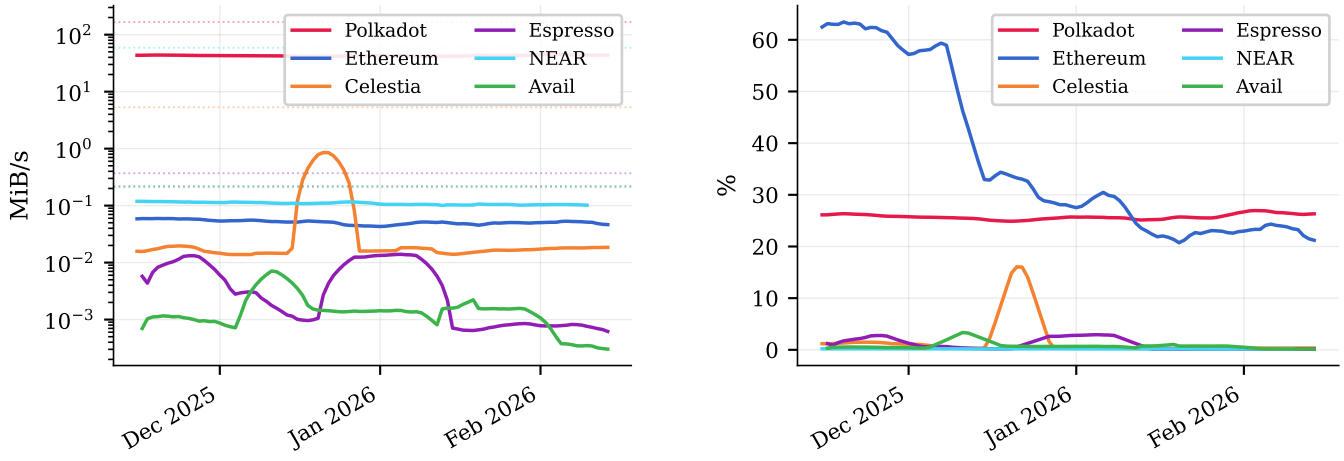
5.4. Natural Experiments: Protocol Upgrades

5.4.1. Ethereum: Pectra → Fusaka → BPO1 → BPO2

Fig. 4 separates the effects of the three upgrades. Under Pectra the blob fee was at the 1-wei minimum on 5–68% of blocks per day. Fusaka's EIP-7918 ties a reserve price to the execution base fee; from December 4 no block is at the floor (Fig. 4(c)) and the median cost rises to ~$0.009/MiB, while utilization stays at ~58%. BPO1 and BPO2 each lowered utilization (from ~58% to ~31% to ~23%) and target utilization (from ~87% to ~47% to ~34%; Fig. 4(b)). The median cost stayed between $0.007 and $0.010/MiB in both BPO eras (Fig. 4(a)), consistent with fees held near the EIP-7918 reserve price instead of being set by congestion. Daily blob volume stayed at ~4.1–4.7 GiB in all four eras, so the added capacity has not yet been used.

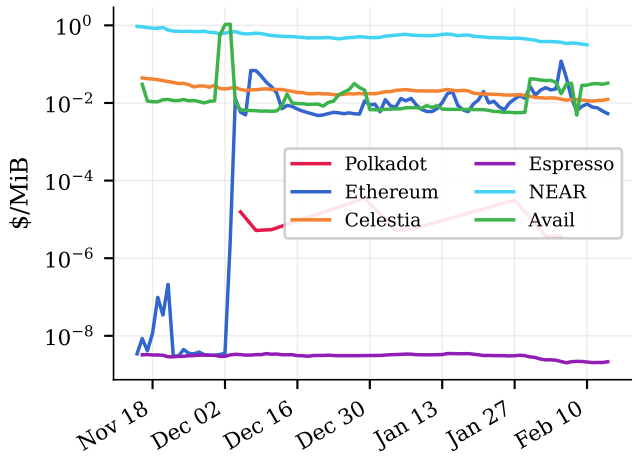
5.4.2. Celestia: December Demand Surge

Celestia raised its maximum square from 128 to 256 on December 3, quadrupling capacity to ~5.3 MiB/s. From December 16 to 21, daily throughput rose eighty-fold, from a baseline

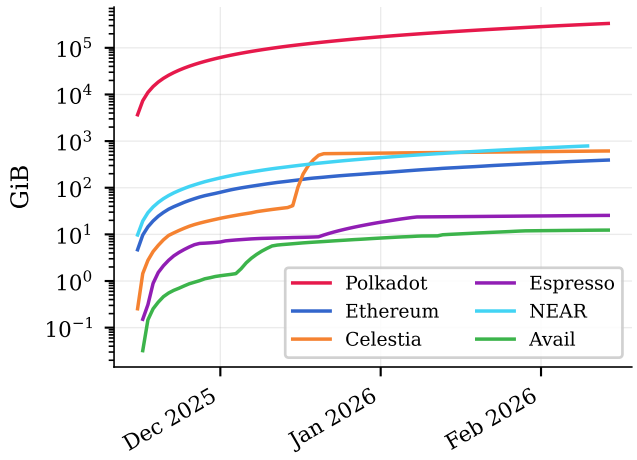


(a) Throughput (MiB/s, 7-day mean, log scale)

(b) Utilization (%, 7-day mean)



(c) Median cost ($/MiB, log scale)



(d) Cumulative volume (GiB, log scale)

Figure 1: Cross-protocol comparison over the 90-day window. All protocols operate well below their capacity ceilings; Celestia's December 16–21 surge peaks at ~22% of its (quadrupled) capacity. Polkadot throughput and volume are upper bounds (Section 4.3).

of ~0.015 MiB/s to ~1.2 MiB/s. That is ~22% of the new ceiling, but it would have saturated the pre-December capacity of 1.33 MiB/s. Cost did not rise: the daily median fell slightly ($0.020 to $0.017/MiB) and the p90 collapsed onto the median, indicating that the surge consisted of large blobs posted at the minimum gas price rather than competing demand. Afterwards throughput and the p90 returned to their earlier levels, so the largest demand event in the window never tested the fee market.

5.5. Block Dynamics and Polkadot Core Engagement

Fig. 5(a–b) shows block time distributions for the two protocols where they are informative; the fixed-cadence protocols (Ethereum 12 s, Polkadot 6 s, Avail 20 s) are omitted. Celestia holds 5–6 s after an elevated p90 in mid-November. Espresso's responsive HotShot consensus has p10 ~1 s and p50 ~2 s, with a p90 that falls from ~6.5 s in the first weeks to ~3 s by February.

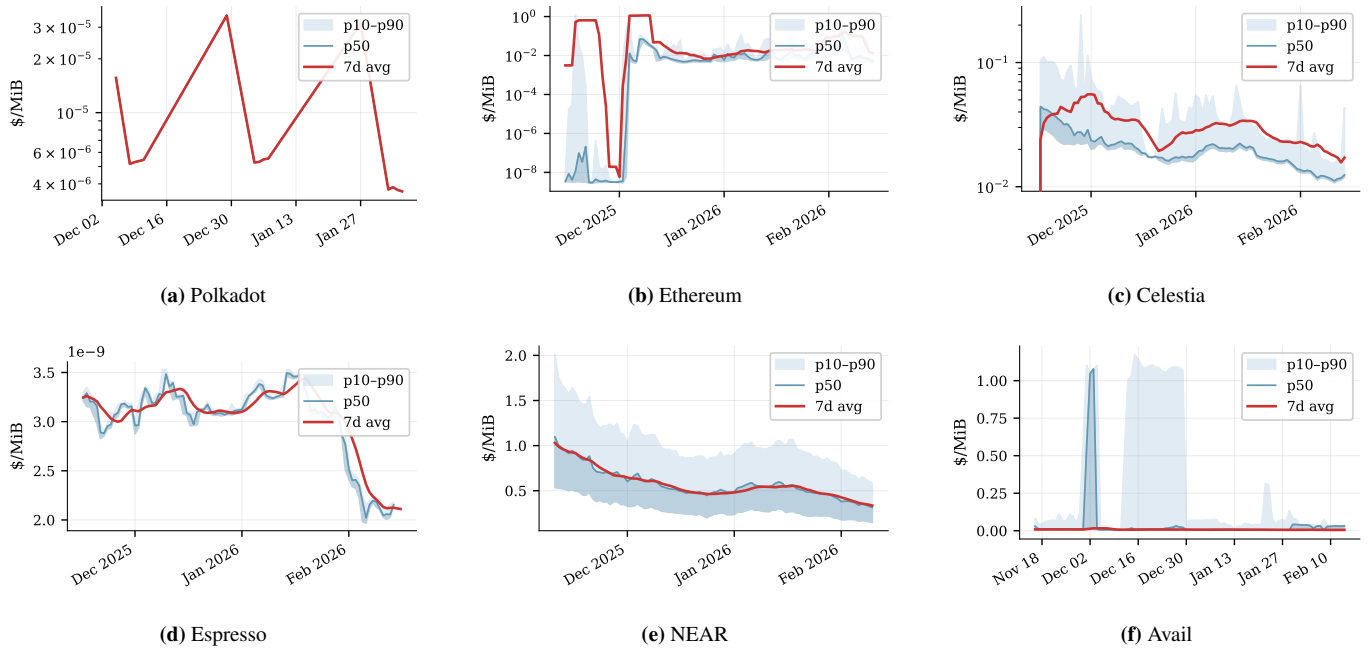
Fig. 5(c) shows that only ~26 of 100 Polkadot cores were engaged on a typical day, leaving ~74% idle capacity: the relay chain could host roughly three times as many parachains at current PoV sizes without parameter changes.

Espresso's payload pattern (two burst periods in November and late December–early January, then collapse to <50 MiB/day) suggests non-organic traffic typical of an early permissioned deployment, in contrast to Celestia's sustained 30–100 MiB/hour baseline.

6. Security and Trust Analysis

6.1. Byzantine Fault Tolerance

All six protocols inherit the classical BFT bound of $f < n/3$, where f is the number of Byzantine-faulty nodes and n is the total committee or validator set size. How the threshold is applied differs between protocols.



(a) Polkadot **(b)** Ethereum **(c)** Celestia

(d) Espresso **(e)** NEAR **(f)** Avail

Figure 2: Daily cost quantile bands ($/MiB) per protocol. (a) Polkadot bulk coretime, on days with purchases only. (b) Ethereum: fees sit at the 1-wei floor until Fusaka (Dec 3), then settle near $0.005–0.02/MiB. (c) Celestia declines from ~$0.05 to ~$0.02/MiB, partly with the TIA price. (d) Espresso's fixed 1 wei/byte fee tracks ETH/USD. (e) NEAR has the highest median among active fee markets. (f) Avail has the widest spread, with p50 spikes above $1/MiB.

Polkadot. Polkadot's relay chain requires $\geq 2/3$ of validators to sign availability bitfields before a parachain block can be included and finalized. With approximately 600 active validators, the system tolerates up to ~200 Byzantine faults. ELVES adds approval checking: randomly selected validators re-execute parachain blocks post-inclusion, and a single invalid execution triggers a dispute that can slash the responsible validators [5]. The probability of an invalid block escaping detection decreases exponentially with the number of approval checkers.

Ethereum. Ethereum's Casper FFG requires a $2/3$ supermajority of validators (by stake weight) to finalize blocks. With approximately 1 million active validators, the economic security backing finality exceeds $30 billion (at ETH ~$3,000). PeerDAS distributes custody across the validator set: each validator holds a subset of blob columns, and system-level availability confidence emerges from the aggregate custody distribution [3].

Celestia. Celestia runs CometBFT (Tendermint) consensus, which requires $\geq 2/3$ of validators (by stake weight) to commit each block. The validator set is capped at 100 in the current configuration. Light-client sampling adds a second check: if validators finalize a block whose data is unavailable, sampling detects the withholding with high probability [12], provided enough light clients take part.

Espresso. Espresso's HotShot consensus operates with 100 nodes across 20 operators in the Mainnet 0 configuration. The DA certificate requires $\geq 2/3$ of committee signatures, tolerating up to ~33 Byzantine nodes. However, the small operator count (20) means that the effective trust assumption is closer to $f < 7$ at the operator level [6].

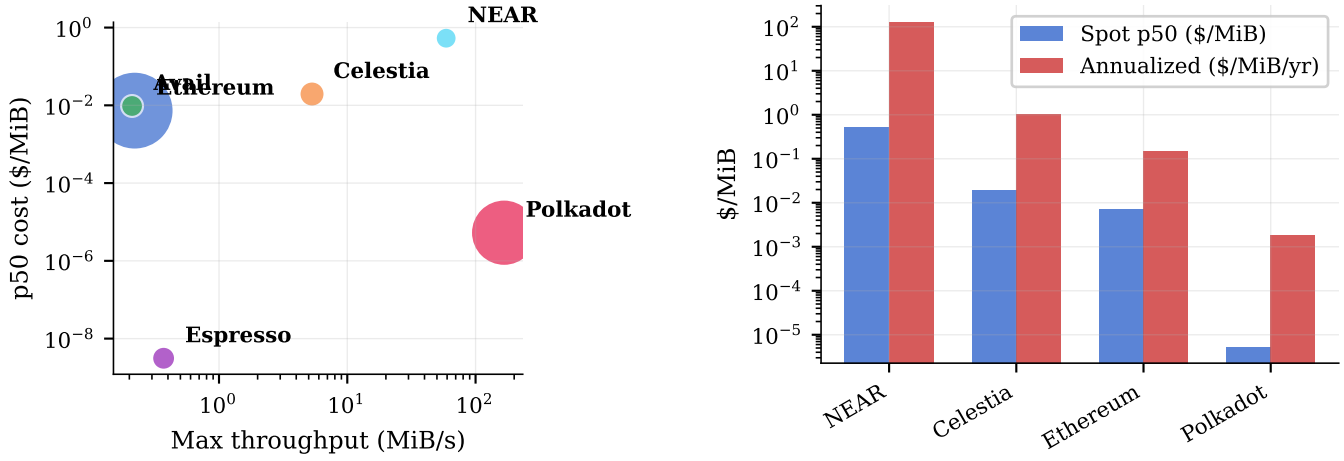
NEAR. NEAR's Nightshade sharding applies the BFT threshold per shard. With 9 shards and approximately 400 validators, the per-shard committee size depends on stake distribution. The protocol assumes that each shard independently maintains $< 1/3$ Byzantine nodes; an adversary who concentrates stake in a single shard can potentially compromise that shard's DA at lower total cost than compromising the network-wide threshold [7]. In practice most mainnet validators currently track all shards, so data is replicated across the network although the protocol only requires shard-local storage, and the per-shard assumption is not yet the binding one.

Avail. Avail's BABE/GRANDPA consensus requires $\geq 2/3$ honest validators for both block production and finality. DAS light clients provide independent verification, analogous to Celestia's model. The VectorX bridge to Ethereum adds a further trust dependency: bridge liveness depends on the prover infrastructure, and the approximately 2-hour bridge delay creates a window during which Avail-native finality and Ethereum-bridged attestations may diverge [8].

Table 8 summarizes the BFT characteristics across protocols.

6.2. *Data Withholding Enforcement*

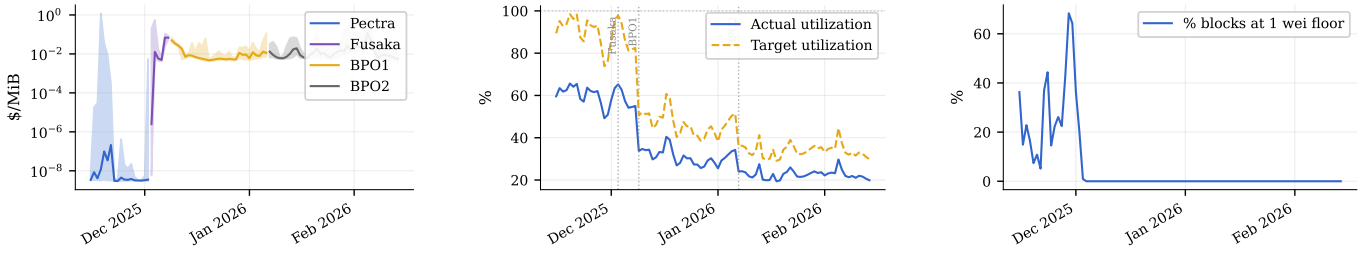
Section 3.3 showed that no protocol slashes for data withholding; here we compare how each one detects it.



(a) Maximum throughput vs. median cost (bubble size: mean utilization)

(b) Spot vs. annualized cost

Figure 3: Capacity, cost and retention. Protocols do not lie on a single capacity–cost frontier, and retention reorders the cost ranking.



(a) Cost bands by era ($/MiB)

(b) Target vs. actual utilization (%)

(c) Blocks at the 1-wei fee floor (%)

Figure 4: Ethereum blob market across the Pectra, Fusaka, BPO1 and BPO2 eras. The fee leaves the floor at Fusaka (EIP-7918), before any capacity change; the BPO forks lower utilization without materially changing median cost.

Detection is sampling-based for Celestia, Avail and Ethereum (missing responses to random samples), attestation-based for Polkadot and NEAR (a missing bitfield or chunk endorsement is visible on-chain, but only validator-level withholding is detected), and certificate-based for Espresso (a missing DA certificate blocks finalization).

Enforcement gap. In every protocol, detecting that data is unavailable does not identify who is responsible. A validator that fails to serve a chunk may be offline, network-partitioned, or maliciously withholding, and the protocol cannot distinguish these cases. This is why no protocol slashes for withholding: slashing on such evidence would also punish honest nodes that have network problems.

6.3. Retention and Persistence

No protocol guarantees permanent availability (Table 3). Four define a window: Polkadot's availability store keeps finalized chunks for 25 hours, NEAR garbage-collects after 5 epochs (~1.5 days), Celestia light nodes sample within 7 days (CIP-036), and Ethereum serves blobs for 4,096 epochs (~18 days). Avail and Espresso define no network-wide expiry: Avail nodes keep block data by default and Espresso nodes choose their own pruning policy, so availability beyond the latest blocks rests on operators rather than the protocol. This matters for rollups. An optimistic rollup needs the DA layer to keep data for at least its challenge period, typically 7 days. Polkadot (25 h) and NEAR (~1.5 d) keep it for less and Celestia's 7-day sampling window leaves no margin, so rollups on these layers must archive data themselves or shorten their challenge periods; on Avail and Espresso it depends on node configuration. Bridges that verify DA attestations must also complete within the retention window; Avail's VectorX attestations reach Ethereum about 2 hours after the block. After expiry, data survives only if full nodes, indexers or rollup operators archive it voluntarily.

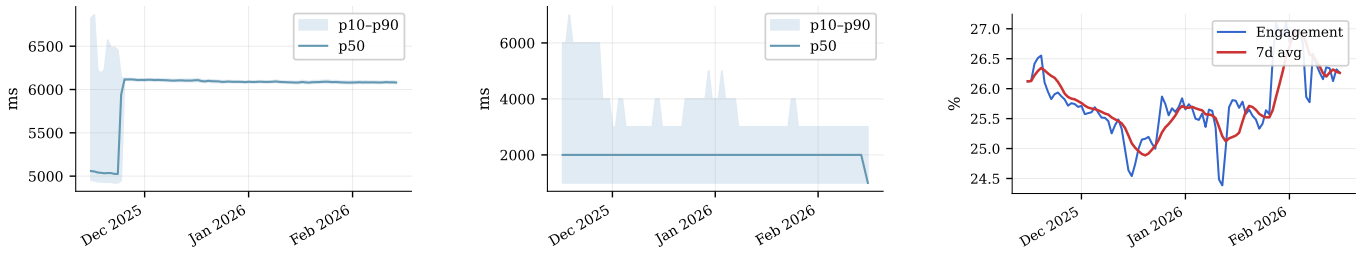
6.4. Trust Assumptions Summary

Table 9 lists each protocol's trust assumptions.

7. Discussion

7.1. Trade-offs Between the Three Paradigms

DAS protocols (Celestia, Avail, Ethereum). These give light clients the strongest guarantees, because a resource-constrained



(a) Celestia block time (ms)

(b) Espresso block time (ms)

(c) Polkadot core engagement (%)

Figure 5: Block time distributions (p10/p50/p90) for Celestia and Espresso, and Polkadot core engagement.

Table 8: Byzantine Fault Tolerance and Validator Set Characteristics

Protocol	BFT Threshold	Validator Count	Consensus	Finality	Stake
Polkadot	$f < n/3$	~600	BABE + GRANDPA	12–60 s	~1.8M DOT min
Ethereum	$f < n/3$ (stake)	~1M	Casper FFG	~12.8 min	32 ETH per validator
Celestia	$f < n/3$ (stake)	~100	CometBFT	Single-slot (~6 s)	Variable
Espresso	$f < n/3$	100 (20 operators)	HotShot	~2 s	None (permissioned)
NEAR	$f < n/3$ per shard	~400	Nightshade	~2 s	Seat-price auction
Avail	$f < n/3$	~100	BABE + GRANDPA	20–60 s	Variable

node can check availability itself. The guarantee weakens if too few nodes sample, a dependence the other two paradigms do not have.

Validator-recovery protocols (Polkadot, NEAR). Polkadot's finality gate (GRANDPA will not finalize blocks without sufficient availability bitfields) is the most direct enforcement we observe: unavailable data cannot be finalized. In exchange, light clients cannot verify availability themselves. For parachain validation this is acceptable, because the relay chain validators are the main consumers of the data; an external rollup using Polkadot or NEAR for DA would have to trust the validator set.

VID protocol (Espresso). None of the other protocols has an equivalent of Tiramisu's algebraic bribery resistance, but in Mainnet 0 it counts for little while the operator set is permissioned (20 operators) and there is no slashing. The 1 wei/byte fee is likewise a placeholder. Whether VID's advantages matter in practice will depend on a permissionless deployment with economic security behind it.

7.2. The Cost–Throughput Frontier

Fig. 3(a) plots median cost against protocol maximum throughput, and there is no simple inverse relationship. Espresso combines low cost with moderate throughput, but its cost is a Mainnet 0 placeholder. Celestia and Avail have moderate throughput at moderate cost. Polkadot has the highest theoretical throughput (~167 MiB/s) at ~$\$5\times10^{-6}$/MiB, although coretime pays for execution as well as DA. NEAR combines high capacity (~59 MiB/s) with the highest cost among active fee markets (~$0.53/MiB), the opposite corner from Polkadot. After Fusaka, Ethereum offers modest capacity at a cost comparable to Celestia and Avail, with the largest validator set and stake behind it.

The protocols were built for different jobs: Polkadot for parachain validation, NEAR for general smart-contract execution, Ethereum for censorship resistance and economic security, and Celestia and Avail as dedicated DA layers. Cost and throughput therefore do not rank them on their own; retention, trust assumptions, light-client verification and ecosystem integration matter as much.

7.3. Polkadot's Scaling Headroom

With only ~26 of 100 cores engaged (Fig. 5(c)), Polkadot has ~74% idle DA capacity: throughput could grow from the current upper bound of ~43 MiB/s toward ~167 MiB/s without parameter changes. As engagement rises, the coretime market should raise core prices; at the current ~$\$5\times10^{-6}$/MiB, cost is unlikely to limit near-term growth. Because actual PoV sizes are not exposed on-chain, a more precise capacity analysis requires PoV-level data.

7.4. Implications for Rollup Developers

Retention constrains a rollup's choice of DA layer before cost does. An optimistic rollup with a 7-day challenge period cannot rely on Polkadot (25 h) or NEAR (~1.5 d) without its own archive, and Celestia's 7-day sampling window leaves no margin. For the same reason, spot prices mislead on short-retention layers; the annualized cost (Section 4.2) counts the reposts needed to keep data available. Fee floors and token prices, more than congestion, currently set what rollups pay: Ethereum's post-Fusaka cost sits near the EIP-7918 reserve price, and Celestia's largest demand surge did not raise its median fee. With every protocol below 40% mean utilization, capacity does not yet separate the options; how fee markets behave under load will matter once utilization rises.

Table 9: Trust Assumption Comparison Across DA Protocols

Protocol	Availability Assumption	Encoding Trust	Additional Assumptions
Polkadot	$\geq 2/3$ validators honest (bitfield voting)	Validators re-encode and verify (approval checking)	Relay chain liveness
Ethereum	$\geq 2/3$ validators honest (Casper FFG) + sufficient custody distribution	KZG algebraic verification (no fraud proofs needed)	Trusted setup integrity (KZG ceremony)
Celestia	$\geq 2/3$ validators honest + sufficient DAS participation	BEFPs from ≥ 1 honest full node	≥ 1 non-eclipsed honest full node
Espresso	$\geq 2/3$ committee signs DA certificate	KZG + vector commitment algebraic verification	Committee honesty; CDN liveness
NEAR	$< 1/3$ Byzantine per shard	Chunk producers re-encode	Honest shard assignment; no stake concentration
Avail	$\geq 2/3$ validators honest + DAS participation	KZG algebraic verification	Trusted setup integrity; VectorX bridge liveness

7.5. Protocol Design Observations

Withholding enforcement. No protocol can attribute withholding to a specific actor, which suggests that slashing for omission faults is harder than slashing for commission faults such as equivocation or invalid execution. Other enforcement models may be needed, for example insurance schemes, reputation systems with gradual stake reduction, or cryptographic mechanisms that make withholding unprofitable without having to attribute it.

Retention. Four protocols define an expiry window, and Avail and Espresso leave retention to node operators. This is by design: keeping all history indefinitely does not scale, and a DA layer exists to provide a window for verification and fraud detection. Rollups and applications that need data beyond that window must store it elsewhere, for example on IPFS, Filecoin or Arweave.

Redundancy and commitments. Among these six protocols, higher redundancy goes with simpler commitments. Celestia (4×) uses NMTs and Polkadot and NEAR (3×) use Merkle trees, while Ethereum and Avail (2×) use KZG, which verifies encoding without fraud proofs. Espresso's polynomial VID also relies on algebraic commitments, which provide encoding verification and bribery resistance at a configurable redundancy.

7.6. Limitations and Threats to Validity

Demand-constrained measurements. Observed throughput reflects demand during one 90-day window; conclusions about maximum throughput must come from protocol parameters (Section 4.3).

Token prices. USD costs depend on token prices, which moved widely during the window (ETH roughly \$2,000–\$4,000, TIA \$0.30–\$0.85), so another window would give different figures. Reporting VWAP and percentiles reduces this effect but does not remove it.

Polkadot throughput upper bound. Polkadot throughput is an upper bound because PoV sizes are not exposed on-chain; parachains typically use 1–3 MiB of the 10 MiB maximum.

Espresso Mainnet 0. Mainnet 0 is a permissioned early deployment, so its cost, throughput and security figures cannot be compared on equal terms with permissionless, economically secured networks. We include it for architectural completeness.

Single observation window. A 90-day window is a snapshot, and behaviour under other market conditions, governance events or adoption levels may differ. The released tooling can be used to extend the window.

Cross-protocol normalization. Converting blob gas, coretime, gas and fixed fees to \$/MiB hides some economic detail: Polkadot's coretime also buys execution, and NEAR's gas price covers general contract execution. The retention-adjusted cost (Table 7, Fig. 3(b)) addresses part of this, but no single metric captures all of it.

8. Conclusion

We measured six production DA protocols (Polkadot ELVES, Ethereum EIP-4844/PeerDAS, Celestia, Espresso Tiramisu, NEAR and Avail) with one methodology, using 90 days of mainnet data covering over 14 million blocks.

8.1. Summary of Findings

Every protocol's mean utilization was below 40%. The networks are demand-constrained, so until they approach saturation, cross-protocol throughput comparisons measure adoption.

Median DA cost spans more than eight orders of magnitude, from ~\$3×10^{-9}/MiB (Espresso, whose 1 wei/byte fee is a Mainnet 0 placeholder) to ~\$0.53/MiB (NEAR). On Ethereum, the blob fee left the 1-wei floor because of Fusaka's EIP-7918 reserve price, before the BPO forks added capacity. Annualizing cost over each protocol's retention window shows that a protocol that is cheap at spot can be expensive when data must stay available.

No protocol slashes for pure data withholding. Polkadot gates finality on availability bitfields, Celestia uses bad-encoding fraud proofs, Ethereum uses peer scoring, Espresso relies on committee reputation, NEAR has an unused challenge

mechanism, and Avail slashes only for equivocation. Attributing withholding, an omission fault, to a specific actor is still unsolved.

No protocol guarantees permanent availability. Protocol-defined retention ranges from 25 hours (Polkadot) to ∼18 days (Ethereum), Avail and Espresso leave retention to node operators, and only Ethereum's window comfortably exceeds a 7-day optimistic-rollup challenge period.

8.2. Future Work

Longitudinal extension: repeating these measurements as utilization rises would show supply-constrained behaviour that the current data cannot. *Stress testing*: controlled synthetic load would measure protocol limits directly, which observational data cannot do. *Withholding enforcement*: incentive-compatible enforcement for omission faults needs formal analysis. *Cross-layer analysis*: measuring end-to-end rollup performance (confirmation time, cost, finality) for each DA backend would show how the choice affects rollups. *PeerDAS at scale*: PeerDAS activated during our window; its effect on node bandwidth and on further BPO increases is the next thing to measure.

Acknowledgements

This work was supported by the Web3 Foundation through the W3F Grants Program.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used AI chat tools to assist with researching approaches to data collection. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] M. B. Saif, S. Migliorini, F. Spoto, A survey on data availability in layer 2 blockchain rollups: Open challenges and future improvements, Future Internet 16 (9) (2024) 315. doi:10.3390/fi16090315.

[2] V. Buterin, D. Feist, D. Loerakker, G. Kadianakis, M. Garnett, M. Taiwo, A. Dietrichs, EIP-4844: Shard blob transactions, Ethereum Improvement Proposals, no. 4844, created 2022-02-25; status Final (2022).
URL https://eips.ethereum.org/EIPS/eip-4844

[3] D. Ryan, D. Feist, F. D'Amato, H.-W. Wang, A. Stokes, EIP-7594: PeerDAS – peer data availability sampling, Ethereum Improvement Proposals, no. 7594, created 2024-01-12; status Final (2024).
URL https://eips.ethereum.org/EIPS/eip-7594

[4] M. Al-Bassam, LazyLedger: A distributed data availability ledger with client-side smart contracts, arXiv preprint arXiv:1905.09274 (2019). arXiv:1905.09274.
URL https://arxiv.org/abs/1905.09274

[5] J. Burdges, A. Cevallos, H. K. Alper, C.-D. Liu-Zhang, F. Shirazi, A. Stewart, R. Habermeier, R. Klotzner, A. Ordian, Efficient execution auditing for blockchains under Byzantine assumptions, Cryptology ePrint Archive, Paper 2024/961 (2024).
URL https://eprint.iacr.org/2024/961

[6] J. Bearer, B. Bünz, P. Camacho, B. Chen, E. Davidson, B. Fisch, B. Fish, G. Gutoski, F. Krell, C. Lin, D. Malkhi, K. Nayak, K. Shen, A. Xiong, N. Yospe, S. Long, The Espresso sequencing network: HotShot consensus, Tiramisu data-availability, and builder-exchange, Cryptology ePrint Archive, Paper 2024/1189 (2024).
URL https://eprint.iacr.org/2024/1189

[7] NEAR Protocol, The NEAR white paper, https://www.near.org/papers/the-official-near-white-paper, accessed: 2026-10-10 (2020).

[8] Avail Project, Avail: The data availability blockchain, Whitepaper, https://www.availproject.org/whitepaper, accessed: 2026-10-10 (2024).

[9] L. T. Thibault, T. Sarry, A. S. Hafid, Blockchain scaling using rollups: A comprehensive survey, IEEE Access 10 (2022) 93039–93054. doi:10.1109/ACCESS.2022.3200051.

[10] L. Heimbach, J. Milionis, The early days of the Ethereum blob fee market and lessons learnt, in: Financial Cryptography and Data Security (FC 2025), Part II, Vol. 15752 of Lecture Notes in Computer Science, Springer, 2026, pp. 53–71. doi:10.1007/978-3-032-07035-7_4.

[11] S. Lee, 180 days after EIP-4844: Will blob sharing solve dilemma for small rollups?, in: 2025 IEEE 45th International Conference on Distributed Computing Systems Workshops (ICDCSW), IEEE, 2025, pp. 487–492. doi:10.1109/ICDCSW63273.2025.00096.

[12] M. Al-Bassam, A. Sonnino, V. Buterin, I. Khoffi, Fraud and data availability proofs: Detecting invalid blocks in light clients, in: Financial Cryptography and Data Security (FC 2021), Part II, Vol. 12675 of Lecture Notes in Computer Science, Springer, 2021, pp. 279–298, preprint: arXiv:1809.09044. doi:10.1007/978-3-662-64331-0_15.

[13] I. S. Reed, G. Solomon, Polynomial codes over certain finite fields, Journal of the Society for Industrial and Applied Mathematics 8 (2) (1960) 300–304. doi:10.1137/0108018.

[14] A. Kate, G. M. Zaverucha, I. Goldberg, Constant-size commitments to polynomials and their applications, in: Advances in Cryptology – ASIACRYPT 2010, Vol. 6477 of Lecture Notes in Computer Science, Springer, 2010, pp. 177–194. doi:10.1007/978-3-642-17373-8_11.

[15] C. Cachin, S. Tessaro, Asynchronous verifiable information dispersal, in: Proceedings of the 24th IEEE Symposium on Reliable Distributed Systems (SRDS'05), IEEE, 2005, pp. 191–202. doi:10.1109/RELDIS.2005.9.

[16] J. Burdges, A. Cevallos, P. Czaban, R. Habermeier, S. Hosseini, F. Lama, H. K. Alper, X. Luo, F. Shirazi, A. Stewart, G. Wood, Overview of Polkadot and its design considerations, arXiv preprint arXiv:2005.13456 (2020). arXiv:2005.13456.
URL https://arxiv.org/abs/2005.13456

[17] P. Jayanthi, T. Wahrstätter, S. Calder-Mason, A. Davis, A. Dietrichs, EIP-7691: Blob throughput increase, Ethereum Improvement Proposals, no. 7691, created 2024-04-17; status Final (2024).
URL https://eips.ethereum.org/EIPS/eip-7691

[18] A. Elowsson, B. Adams, F. D'Amato, EIP-7918: Blob base fee bounded by execution cost, Ethereum Improvement Proposals, no. 7918, created 2025-03-25; status Final (2025).
URL https://eips.ethereum.org/EIPS/eip-7918

[19] K. Croman, C. Decker, I. Eyal, A. E. Gencer, A. Juels, A. Kosba, A. Miller, P. Saxena, E. Shi, E. G. Sirer, D. Song, R. Wattenhofer, On scaling decentralized blockchains, in: Financial Cryptography and Data Security: FC 2016 International Workshops, BITCOIN, VOTING, and WAHC, Vol. 9604 of Lecture Notes in Computer Science, Springer, 2016, pp. 106–125. doi:10.1007/978-3-662-53357-4_8.

[20] M. Hall-Andersen, M. Simkin, B. Wagner, Foundations of data availability sampling, IACR Communications in Cryptology 1 (4) (2025). doi:10.62056/a09qudhdj.

[21] M. Hall-Andersen, M. Simkin, B. Wagner, FRIDA: Data availability sampling from FRI, in: Advances in Cryptology – CRYPTO 2024, Part VI, Vol. 14925 of Lecture Notes in Computer Science, Springer, 2024, pp. 289–324. doi:10.1007/978-3-031-68391-6_9.

[22] M. Yu, S. Sahraei, S. Li, S. Avestimehr, S. Kannan, P. Viswanath, Coded Merkle tree: Solving data availability attacks in blockchains, in: Financial Cryptography and Data Security (FC 2020), Vol. 12059 of Lecture Notes in Computer Science, Springer, 2020, pp. 114–134. doi:10.1007/978-3-030-51280-4_8.

[23] P. Sheng, B. Xue, S. Kannan, P. Viswanath, ACeD: Scalable data availability oracle, in: Financial Cryptography and Data Security (FC 2021), Part II, Vol. 12675 of Lecture Notes in Computer Science, Springer, 2021, pp. 299–318. doi:10.1007/978-3-662-64331-0_16.

[24] K. Nazirkhanova, J. Neu, D. Tse, Information dispersal with provable retrievability for rollups, in: Proceedings of the 4th ACM Conference on Advances in Financial Technologies (AFT '22), ACM, 2022, pp. 180–197. doi:10.1145/3558535.3559778.

[25] B. Fisch, A. Lazzaretti, Z. Liu, L. Yang, Permissionless verifiable information dispersal (data availability for Bitcoin rollups), in: 2025 IEEE Symposium on Security and Privacy (SP), IEEE, 2025, pp. 1983–2001. doi:10.1109/SP61157.2025.00248.

[26] ethereum.org, Danksharding, https://ethereum.org/roadmap/danksharding/, accessed: 2026-10-10 (2024).

[27] A. Chaudhuri, S. Basak, C. Kiraly, D. Ryajov, L. Bautista-Gomez, On the design of Ethereum's data availability sampling: A comprehensive simulation study, in: Proceedings of the 6th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS), IEEE, 2024, pp. 1–4, extended version: arXiv:2407.18085. doi:10.1109/BRAINS63024.2024.10732486.

[28] M. Pigaglio, O. Ascigil, M. Król, F. Lange, K. Peeroo, S. Rene, R. Sadre, V. Stankovic, E. Rivière, PANDAS: Peer-to-peer, adaptive networking allowing data availability sampling within Ethereum consensus timebounds, in: Proceedings of the 26th International Middleware Conference (Middleware '25), ACM, 2025, pp. 167–179. doi:10.1145/3721462.3770769.

[29] D. Crapis, E. W. Felten, A. Mamageishvili, EIP-4844 economics and rollup strategies, in: Financial Cryptography and Data Security. FC 2024 International Workshops, Vol. 14746 of Lecture Notes in Computer Science, Springer, 2024, pp. 135–149. doi:10.1007/978-3-031-69231-4_10.

[30] S. Park, B. Mun, S. Lee, W. Jeong, J. Lee, H. Eom, H. Jang, Impact of EIP-4844 on Ethereum: Consensus security, Ethereum usage, rollup transaction dynamics, and blob gas fee markets, IEEE Transactions on Network Science and Engineering 13 (2026) 1294–1310. doi:10.1109/TNSE.2025.3594281.

[31] Polkadot Developer Docs, Overview of the Polkadot relay chain, https://docs.polkadot.com/reference/polkadot-hub/consensus-and-security/relay-chain/, accessed: 2026-10-10 (2025).